

Actuarial Neural Networks and Uncertainty

3rd Joint Conference on Statistical and Data Science

Patrick Laub (罗新), UNSW

Joint work with Benjamin Avanzi, **Eric Dong**, and Bernard Wong

Outline

- **Distributional Regression**
- Current solutions
- Distributional Refinement Network

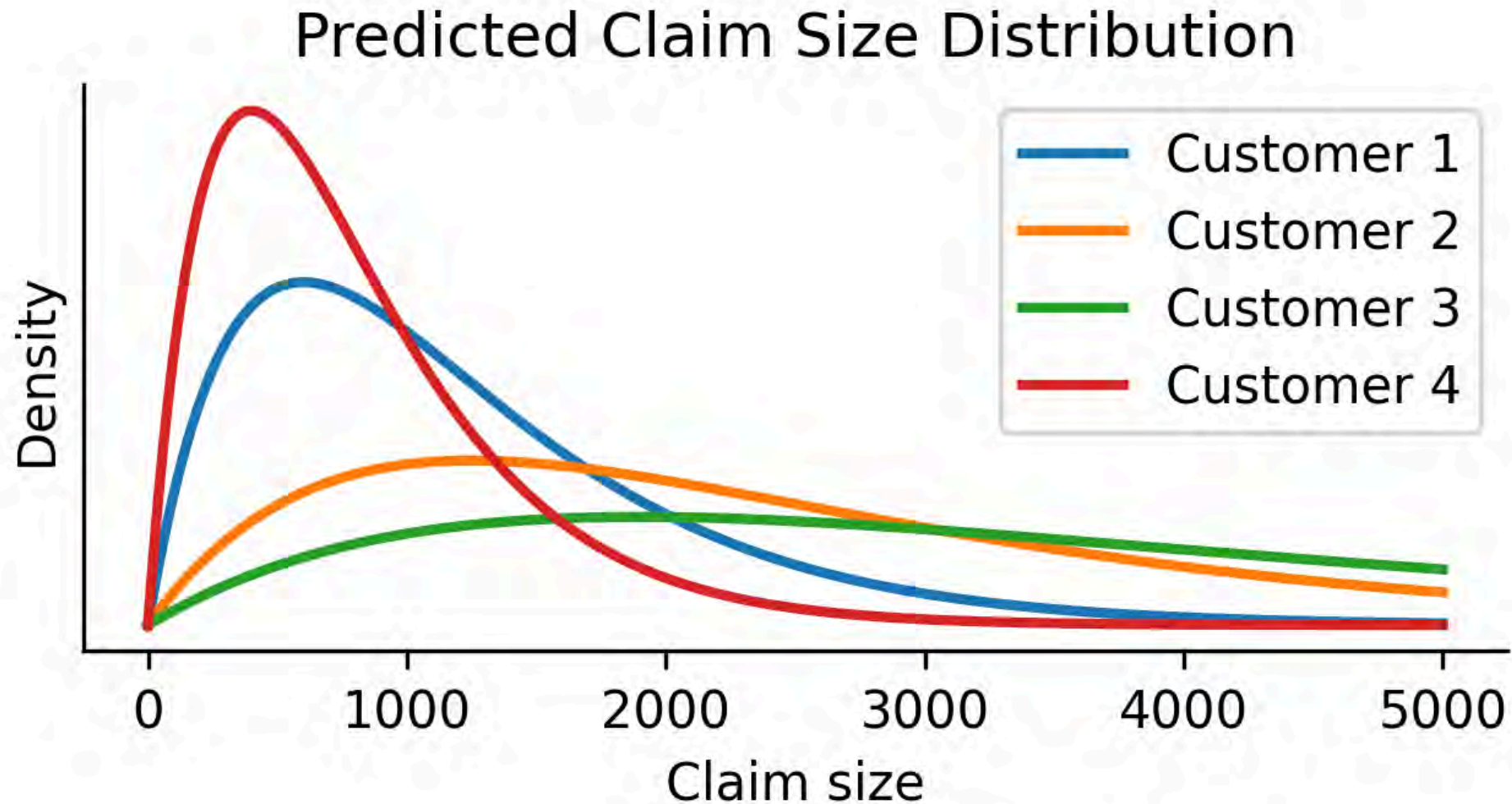
Claim size prediction

“We determine how much an insurance company should charge for auto insurance, taking into account many factors such as the car that is being insured and details about the driver.”

 Age	 Age	 Type	Cost
25	3	 Sedan	\$1,200
40	5	 SUV	\$2,500
19	1	 Sports Car	\$3,800
60	10	 Hatchback	\$800

→

Distributional regression



Actuaries avoid neural networks

They're not *inherently interpretable*, so just have to look at inputs and outputs from the black box.

NN models typically just output a prediction without any sense of its *confidence*.

Therefore actuaries cannot *trust* the neural network (NN) models, which is a **dealbreaker**.

Outline

- Distributional Regression
- **Current solutions**
- Distributional Refinement Network

A generalised linear model

E.g., a gamma GLM with a log link function: $Y|\mathbf{X} \sim \text{Gamma}(k, \mu(\mathbf{X}))$

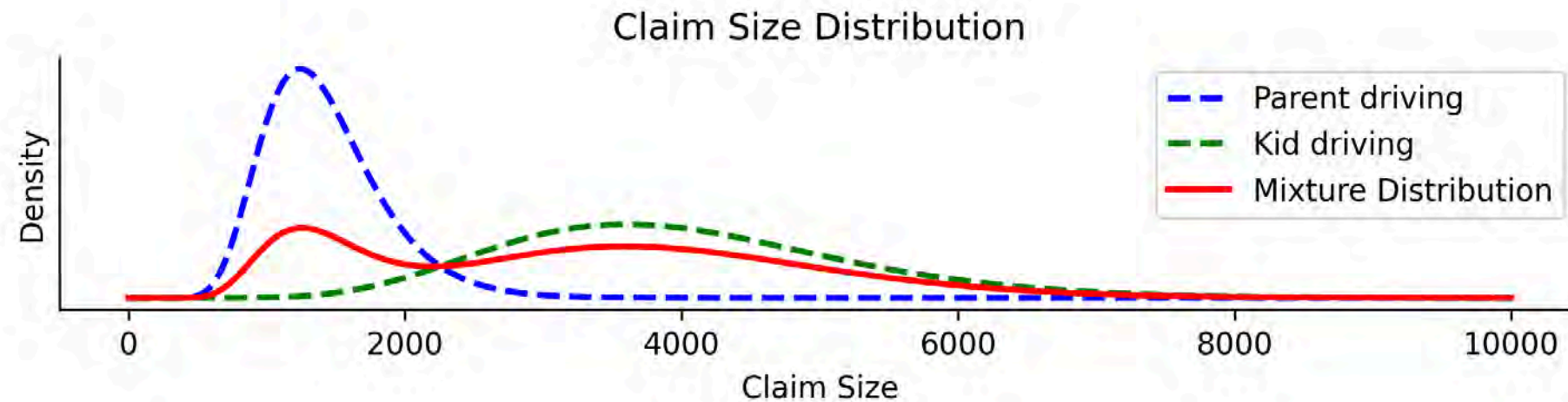
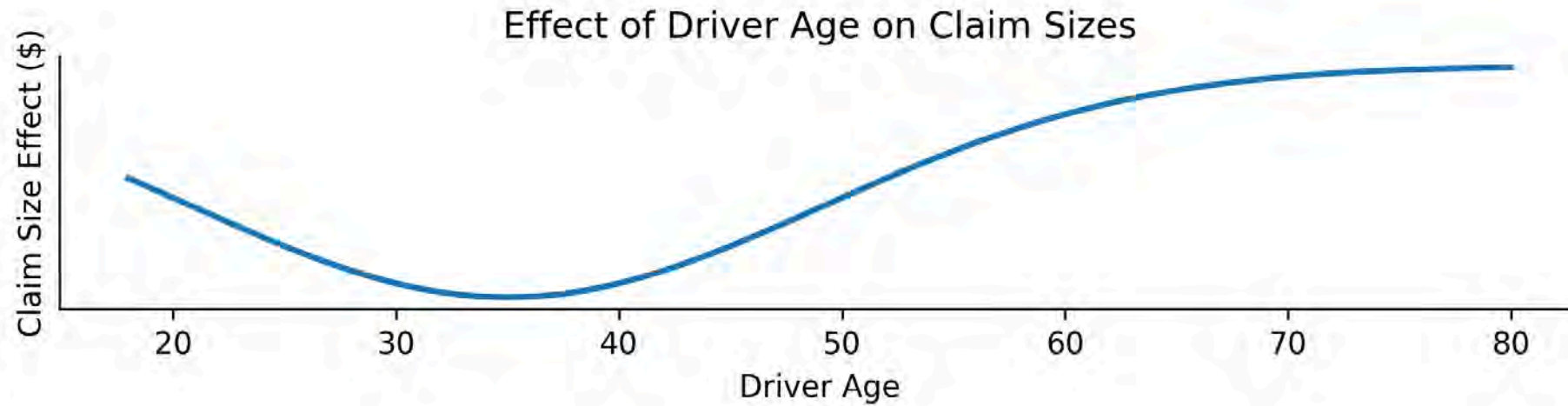
$$\mathbb{E}[Y|\mathbf{X} = \mathbf{x}] = \mu(\mathbf{x}) = \exp\{\langle \boldsymbol{\beta}, \mathbf{x} \rangle\}$$

$$\mathbb{V}[Y|\mathbf{X} = \mathbf{x}] = \mu(\mathbf{x})^2 / k.$$

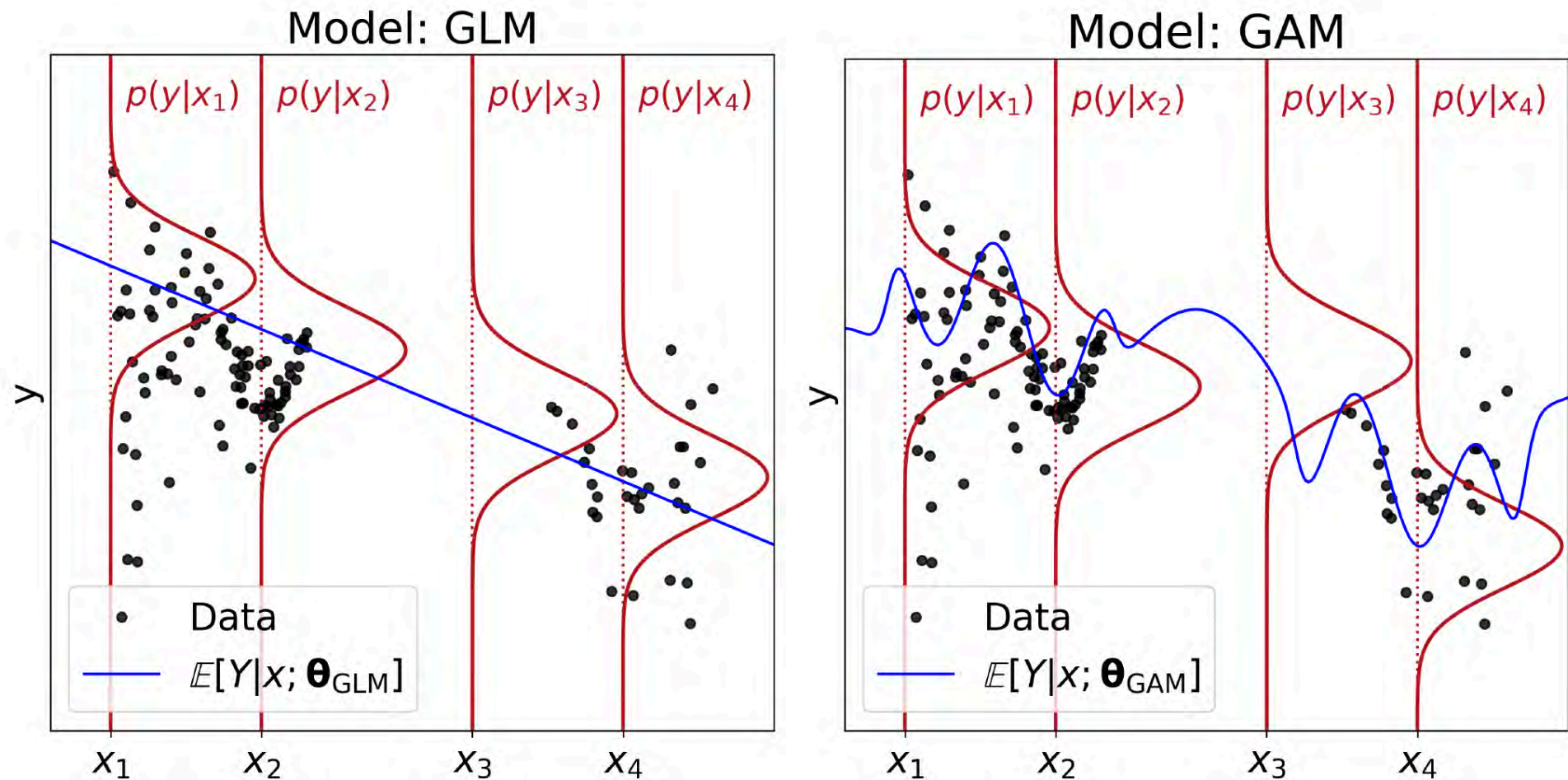
A simple model, easy to train and interpret, but GLMs can be

1. Bad at *regression* (the process of transforming covariates into a mean prediction)
2. Bad at *distributional* regression (being able to make any distribution as an output)

Examples



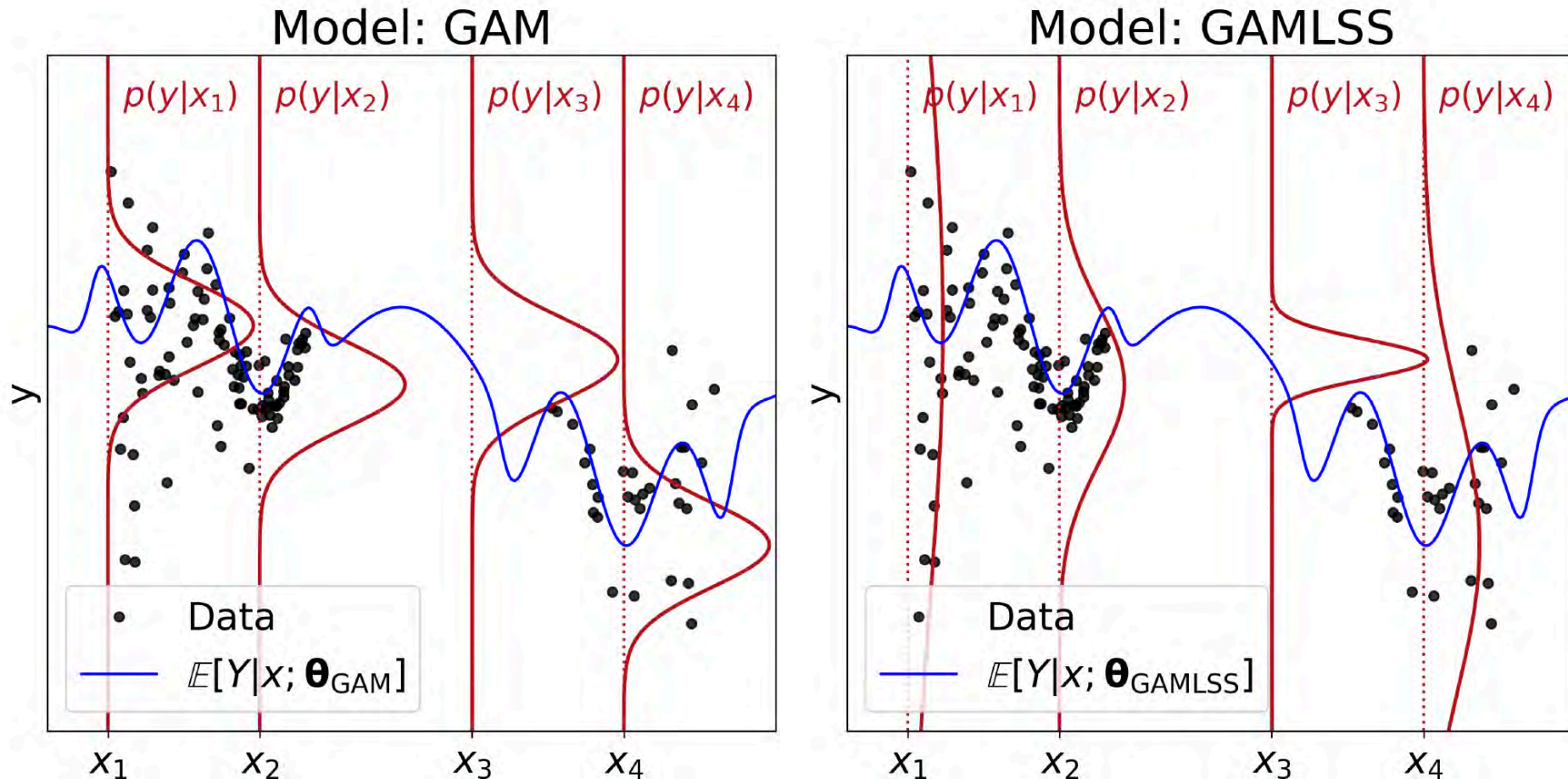
Generalised Additive Models?



Mean estimation is more flexible, but predicted distributions are still rigid.

Thanks to Eric Dong for this figure and the next one.

GAM for Location, Scale, and Shape?



Improved regression and *distributional* regression. Still constrained by parametric assumptions.

Reference: Rigby & Stasinopoulos (2005)

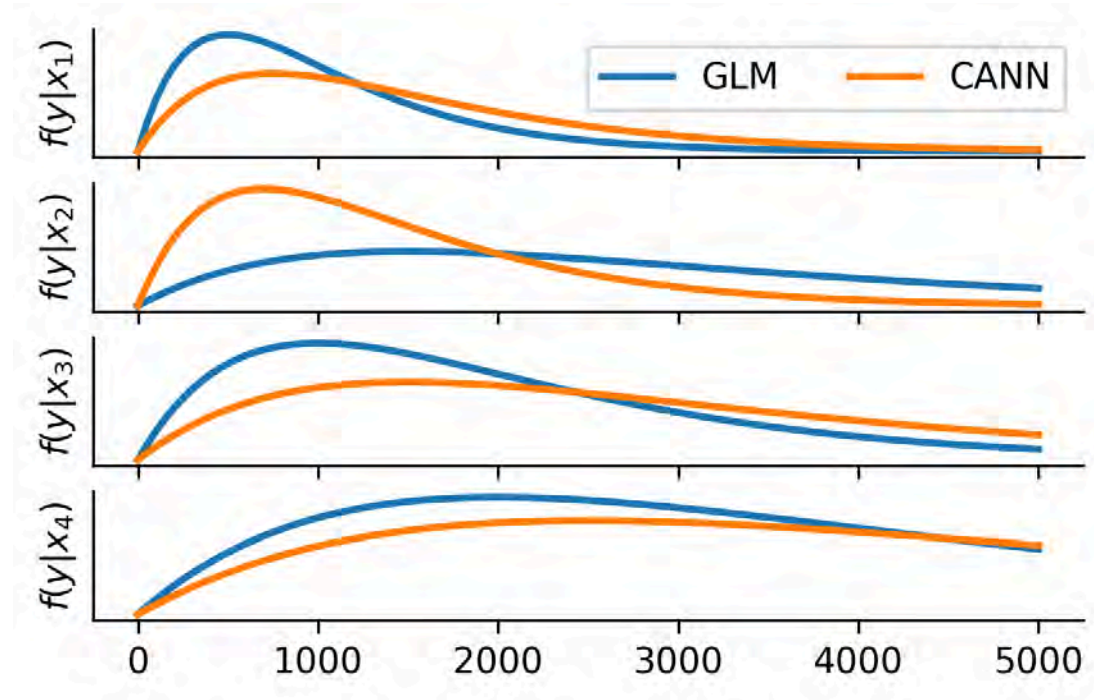
Combined Actuarial Neural Network

1. Fit a GLM with

$$\mu(\mathbf{x}) = g^{-1}(\langle \boldsymbol{\beta}, \mathbf{x} \rangle)$$

2. Fit a neural network that creates offsets $s(\mathbf{x})$ and predictions of

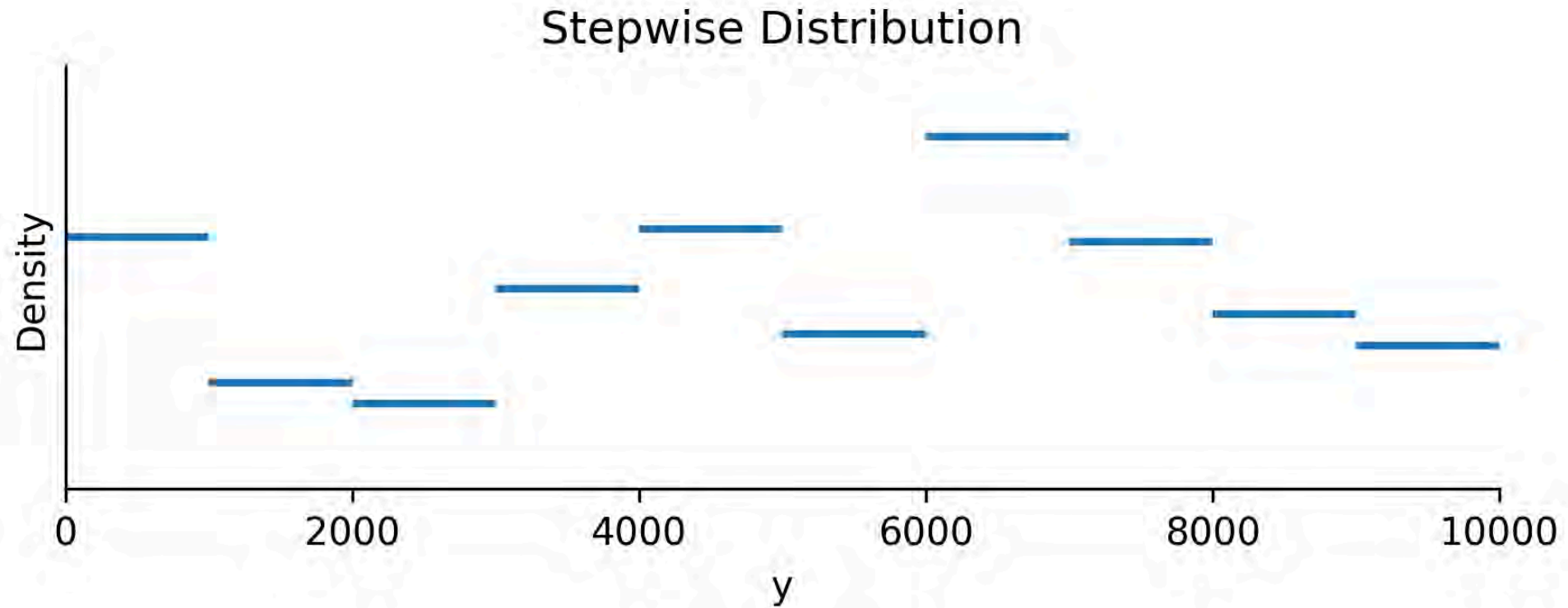
$$\mu(\mathbf{x}) = g^{-1}(\langle \boldsymbol{\beta}, \mathbf{x} \rangle + s(\mathbf{x})).$$



This makes the ‘regression’ part smarter, but not the ‘distribution’ part

Deep Distributional Regression

Basically a neural network that outputs a non-parametric distribution:



Reference: Li et al. (2021)

DDR: Pros and cons

Pros

- Maximum flexibility in transforming the inputs (NN)
- Maximum flexibility in the output distribution (non-parametric)

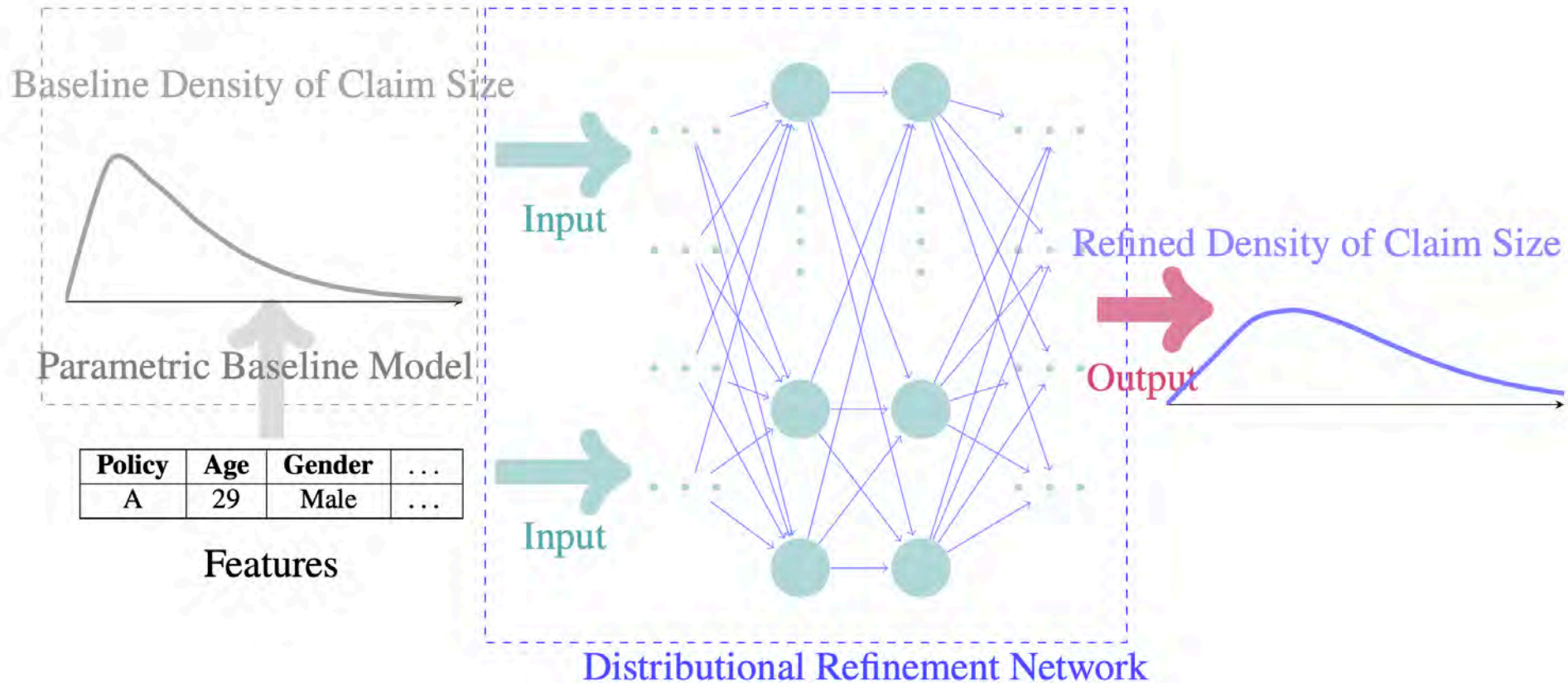
Cons

- Uninterpretable
- Choosing the cutpoints is more work
- Can only model distributions with finite support
- Have one single shared max value for all output distributions

Outline

- Distributional Regression
- Current solutions
- **Distributional Refinement Network**

Distributional Refinement Network

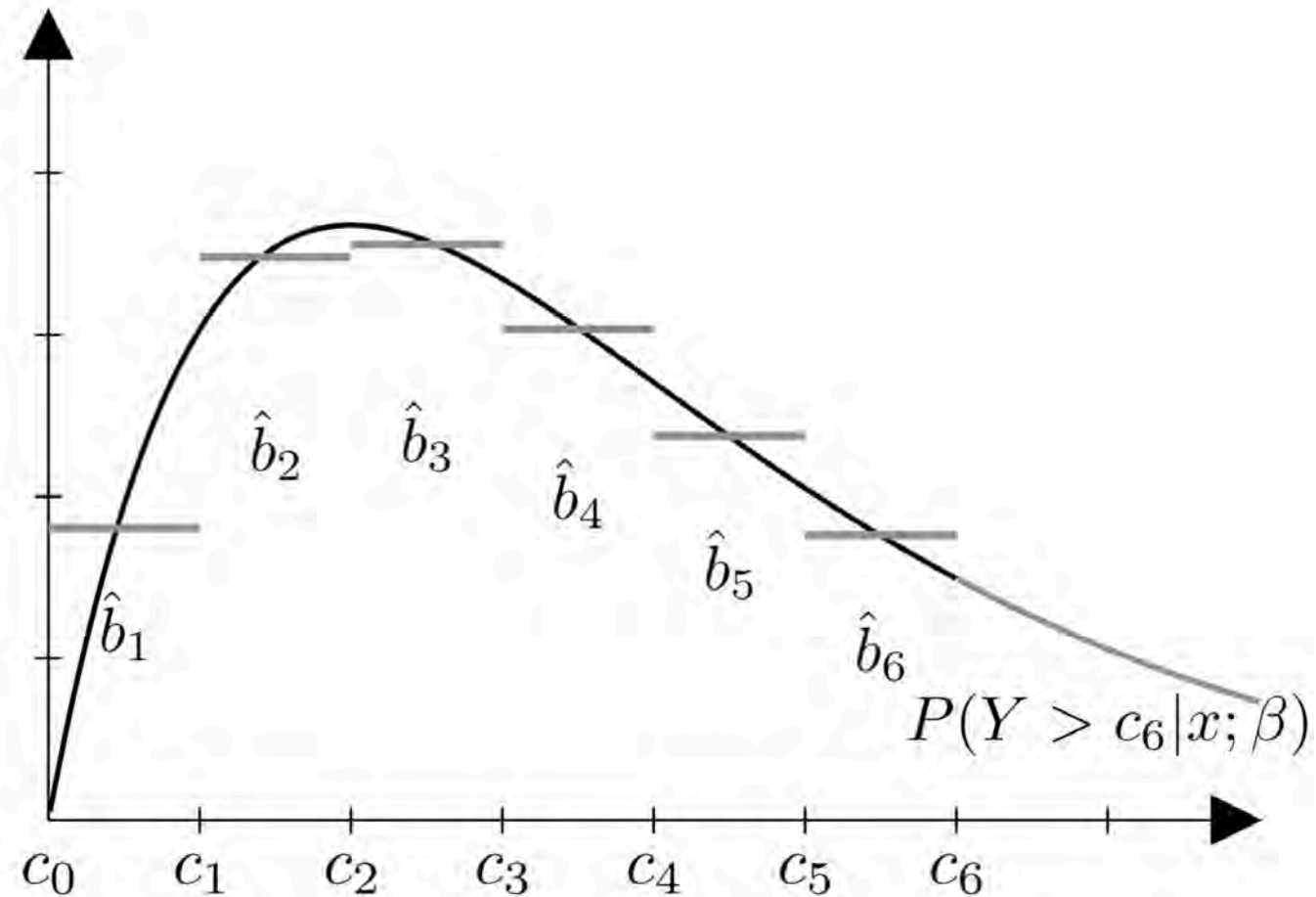


DRN first uses a trusted baseline model, then makes small adjustments to it

Thanks to Eric Dong!

Baselines

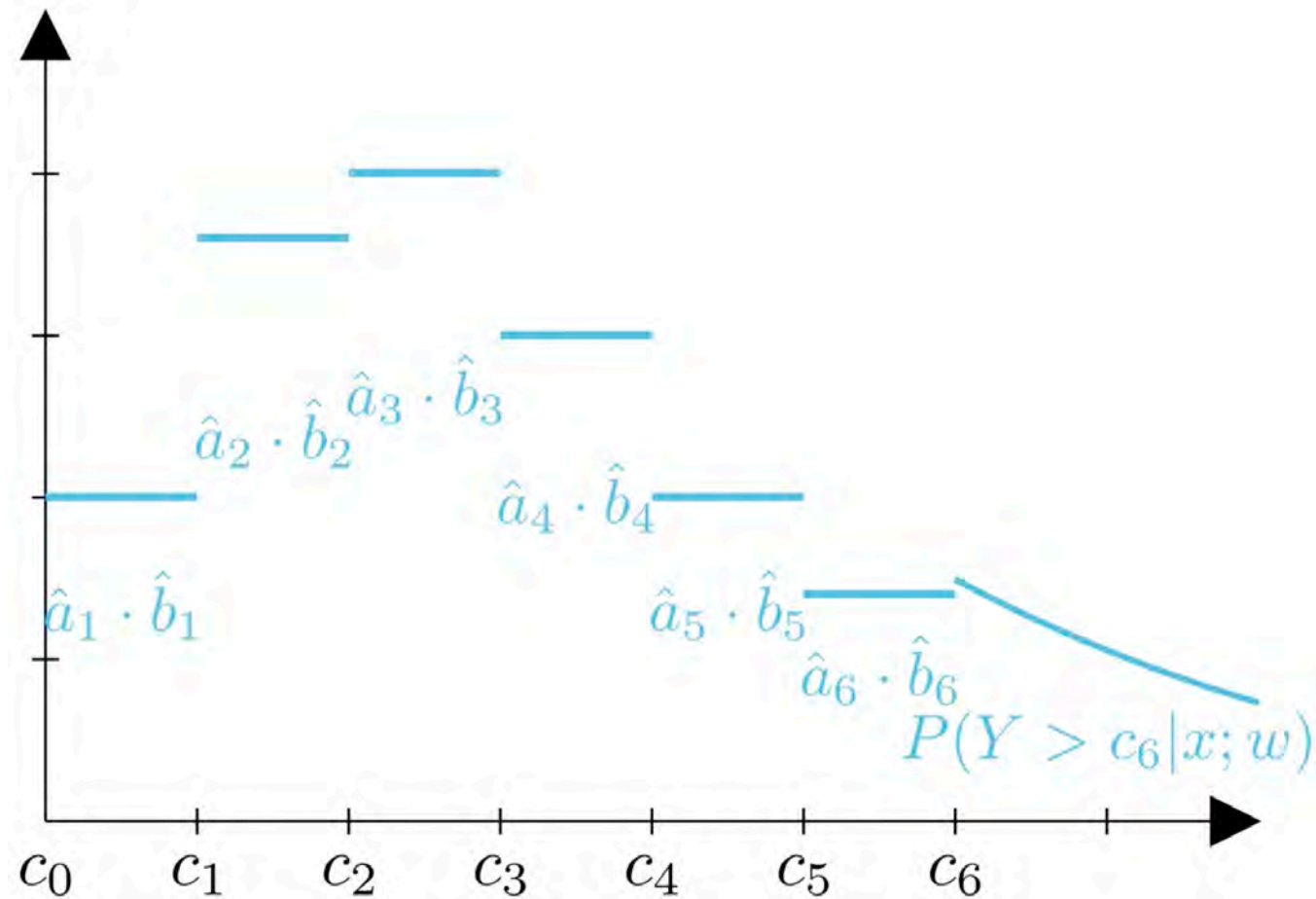
Add in a baseline model, and “discretise” it



Baseline Probability Masses

Adjustments

Then adjust the heights of the bins with NN:



DRN output distribution

Loss and Regularisation

We minimise

$$\begin{aligned} \text{Loss} = & \text{Distributional Accuracy (e.g. NLL)} \\ & + \text{Baseline Resemblance (e.g. KL Div.)} \\ & + \text{Density Smoothness (e.g. Second-Order Difference)} \end{aligned}$$

‘Accuracy’ for distributional regression

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be the dataset for evaluation and $\mathbf{w}$ represent the trained parameters/weights of the model.

The two main evaluation metrics are *continuous ranked probability score*,

$$\text{CRPS}(\mathcal{D}, \mathbf{w}) = \frac{1}{n} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}} \int_y (F_{Y|\mathbf{X}}(y|\mathbf{x}_i; \mathbf{w}) - \mathbf{1}_{\{y > y_i\}})^2 dy,$$

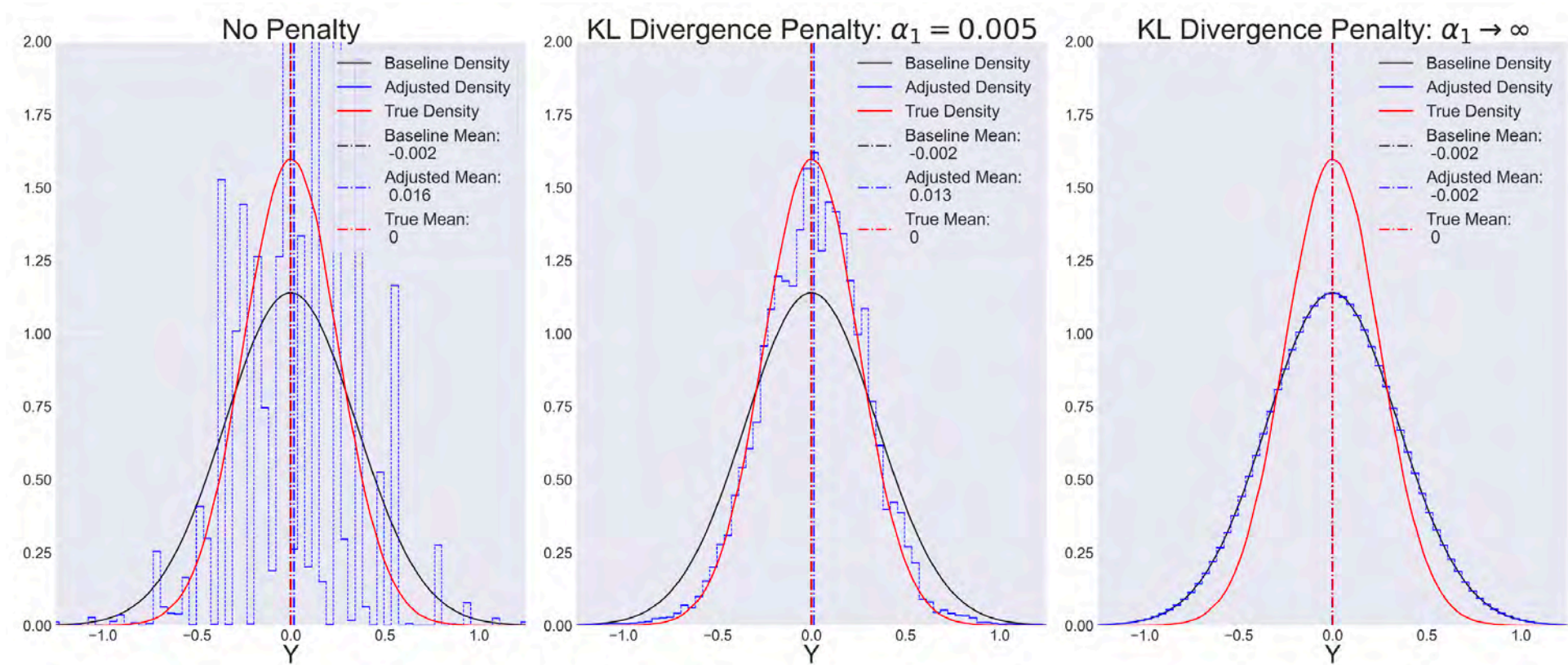
and *negative log-likelihood*,

$$\text{NLL}(\mathcal{D}, \mathbf{w}) = -\frac{1}{n} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}} \ln f_{Y|\mathbf{X}}(y_i|\mathbf{x}_i; \mathbf{w}).$$

These are *proper scoring rules* (Gneiting & Raftery, 2007).

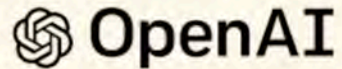
Faithfulness to baseline

We have a *lever* to control how far the NN can deviate from a *trusted inherently interpretable baseline*.



This is used in ChatGPT

State-of-the-art LLMs still use this approach



“Following Jaques et al. (2017; 2019), we use a KL constraint to prevent the fine-tuned model from drifting too far from the pretrained model.”

- [Fine-Tuning Language Models from Human Preferences](#) (2019)
- [Learning to summarize from human feedback](#) (2020)
- [InstructGPT / ChatGPT](#) (2022)
- [Direct Preference Optimization \(DPO\)](#) (2024)



$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathcal{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \quad (1)$$

$$\mathcal{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1, \quad (2)$$

Social RL - Natasha Jaques

35

The KL term in AI chat models

Natasha Jaques (2025), [What Makes ChatGPT Chat? Modern AI for the layperson](#)



Code: Training DRN

Loaded the datasets already, then train the GLM and the DRN:

```
from drn import GLM, DRN, CANN, MDN, DDR

glm_model = GLM.from_statsmodels(X_train, y_train, distribution="gamma")

drn_model = DRN(glm_model, hidden_size=256, num_hidden_layers=2)
drn_model.fit(X_train, y_train, X_val, y_val, epochs=100, patience=5)

print(f"y_train min: {y_train.min().item()}, y_train max: {y_train.max().item()}")
print(drn_model.cutpoints)
```

```
y_train min: 0.0011782477341389, y_train max: 163.427013897281
[0.00, 1.30, 2.60, 3.90, 5.20, 6.50, 7.80, 9.10, 10.40, 11.70, 13.00, 14.30, 15.60, 16.90, 18.20,
19.50, 20.80, 23.40, 24.70, 26.00, 27.30, 29.90, 31.20, 32.50, 55.90, 131.30, 156.00, 171.60]
```

Training CANN, MDN, DDR:

```
cann_model = CANN(glm_model)
cann_model.fit(X_train, y_train, X_val, y_val, epochs=100, patience=5)

mdn_model = MDN(X_train.shape[1])
mdn_model.fit(X_train, y_train, X_val, y_val, epochs=100, patience=5)

ddr_model = DDR(X_train.shape[1], drn_model.cutpoints)
ddr_model.fit(X_train, y_train, X_val, y_val, epochs=100, patience=5)
```


Code: Distributional forecasts

```
glm_model.distributions(x)
```

```
Gamma(concentration: tensor([0.30]), rate: tensor([0.14]))
```

```
cann_model.distributions(x)
```

```
Gamma(concentration: tensor([0.29]), rate: tensor([0.14]))
```

```
mdn_model.distributions(x)
```

```
MixtureSameFamily(
  Categorical(probs: torch.Size([1, 5]), logits: torch.Size([1, 5])),
  Gamma(concentration: torch.Size([1, 5]), rate: torch.Size([1, 5])))
```

```
ddr_model.distributions(x)
```

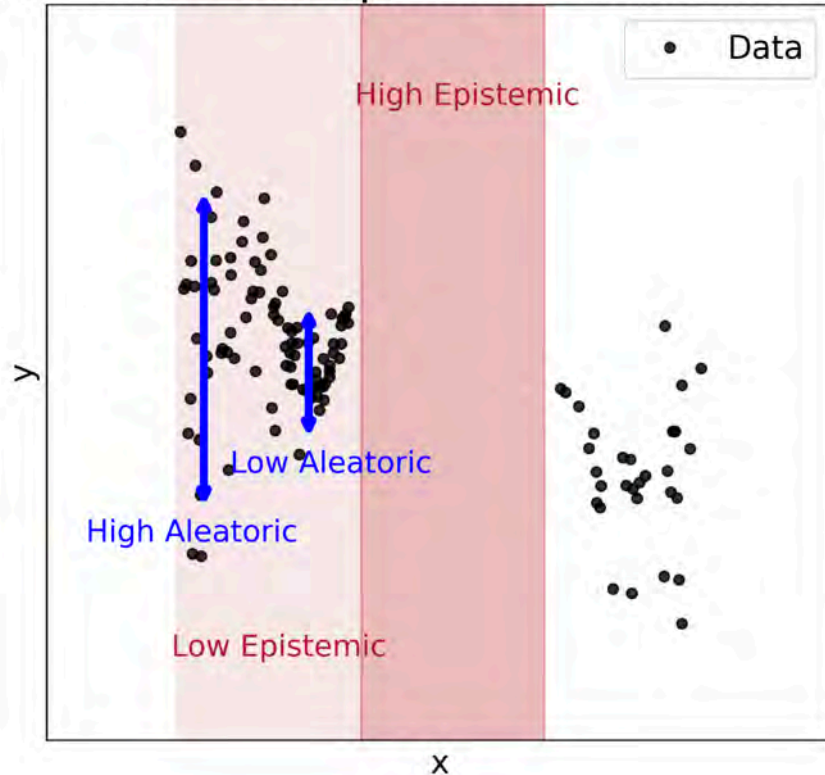
```
Histogram(cutpoints: torch.Size([28]), prob_masses: torch.Size([1, 27]))
```

```
drn_model.distributions(x)
```

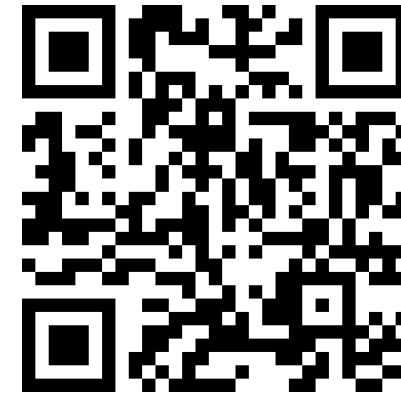
```
ExtendedHistogram(baseline: Gamma(concentration: tensor([0.30]), rate: tensor([0.14])), cutpoints:
torch.Size([28]), prob_masses: torch.Size([1, 27]))
```

Next steps & Conclusion

Aleatoric vs. Epistemic Uncertainty



- More than just mean predictions
- Checkout Eric's [drn](#) package on pypi (major update coming soon)
- Suggestions and questions welcome, thanks for your attention!



How to capture epistemic uncertainty? What to do if you had both of these?

The arXiv link

Patrick Laub <https://laub.au/>

References

- Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359–378.
- Li, R., Reich, B. J., & Bondell, H. D. (2021). Deep distribution regression. *Computational Statistics & Data Analysis*, 159, 107203.
- Nelder, J. A., & Wedderburn, R. W. M. (1972). Generalized linear models. *Journal of the Royal Statistical Society. Series A (General)*, 135(3), 370–384. <http://www.jstor.org/stable/2344614>
- Rigby, R. A., & Stasinopoulos, D. M. (2005). Generalized additive models for location, scale and shape. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 54(3), 507–554.
- Schelldorfer, J., & Wüthrich, M. V. (2019). Nesting classical actuarial models into neural networks. *SSRN*.